



QUEST OF THE CODE REALM

A 2-Day Capstone Project — Build Your Own Text Adventure Game in Python

Built for graduates of CodeHer Workshops 1–3

`print()` • variables • `input()` • `if / elif / else` • `for & while` loops

Welcome, adventurer.

You've learned the basic spells of Python: `print()`, variables, `input()`, conditionals, and loops. Now it's time to combine all of them into one real project — not a 10-line script, but something with multiple files of logic, a real structure, and a genuine "I built this" feeling at the end.

This is QUEST OF THE CODE REALM: a text-based fantasy adventure game, played entirely in the terminal, where the player creates a hero, explores a kingdom, fights monsters, manages stats, and faces a final boss. Every single mechanic uses **ONLY** the tools from Workshops 1–3. Nothing here is beyond what you already know — you're just going to combine it in a bigger, more satisfying way.

Budget about 2 full working days (roughly 10–14 hours total). Go slower if you want — a real project always takes longer than you think, and that's normal!



Quest Log (What's Inside)

Think of this like a level map. Each day is a region of the kingdom; each step is a screen you have to clear before moving to the next one.

Section	Title	What you'll practice
Part 0	Setting Up Your Quest	Folder structure, planning your variables
Day 1 — Part 1	Character Creation	Hero name, class, starting stats
Day 1 — Part 2	The Village & First Choices	Branching paths with <code>if/elif/else</code>
Day 1 — Part 3	The Forest Encounter	Your first random-ish monster fight

Day 1 — Milestone	Checkpoint Save	Test everything before moving on
Day 2 — Part 4	The Inventory & Shop	Loops for managing gold and items
Day 2 — Part 5	The Trial Gauntlet	A while loop battle system with HP
Day 2 — Part 6	The Final Boss	Combine everything into one big fight
Day 2 — Part 7	The Ending & Polish	Multiple endings + replay loop
Bonus Quests	Stretch Goals	Optional extra challenges if you finish early

Part 0 — Setting Up Your Quest

Before writing a single line of game logic, set up your workspace properly. This alone is good practice for every future project you'll build.

STEP 0.1 Create a project folder

Create a new folder called `quest_of_the_code_realm`. This is where every file for this project will live.

HINT

Keep this separate from your workshop notebook files. A clean folder = a clean brain. You can use a single `.py` file (like `quest.py`) for this whole project — no need for multiple files unless you want to organize later as a bonus.

STEP 0.2 Plan your game on paper first

Before coding, sketch out (on paper or in a notes app):

- What stats your hero will have (e.g. HP, gold, strength)
- What classes a player can choose (e.g. Warrior, Mage, Rogue)
- A rough story order: village → forest → trial → boss → ending

HINT

This step feels slow but it isn't optional. Every real developer plans before typing. 10 minutes of planning saves an hour of confused debugging later.

STEP 0.3 Create `quest.py` and add your game's title screen

Use `print()` to display an ASCII-art-ish title screen and a short intro paragraph explaining the game to the player.

```
print("=====")
print("      QUEST OF THE CODE REALM")
print("=====")
print("")
print("A great darkness stirs beyond the village.")
print("Only a hero brave enough to learn its name")
print("can hope to stop it.")
print("")
```

HINT

Run this constantly! Get used to running your file after every small change (`python quest.py`) — small, frequent checks catch bugs fast.

Part 0 Checkpoint

Before moving on, confirm: you have a `quest.py` file, it runs without errors, and it prints your title screen.

Day 1 — Character Creation, the Village, and the First Fight

DAY 1 GOAL: By the end of today, your player can create a hero, make a meaningful choice in the village, and survive (or lose) one battle.

Part 1 — Character Creation

This is where variables and `input()` come together to make the player feel like the game is actually theirs.

STEP 1.1 Ask for the hero's name

Use `input()` to ask the player for their hero's name, and store it in a variable called `hero_name`.

```
hero_name = input("What is your hero's name? ")
print("")
print("Welcome, " + hero_name + "!")
```

HINT

Remember: `input()` always gives back a string. You don't need `int()` here since a name isn't a number.

STEP 1.2 Let the player choose a class

Print 3 class options (Warrior, Mage, Rogue) and use `input()` to capture the player's choice into a variable called `hero_class`.

HINT

This is just like the calculator project from Workshop 2 — print the menu, then store what the player types.

STEP 1.3 Use `if / elif / else` to set starting stats based on class

Each class should start with different stats. For example:

- Warrior → high HP, low Mage Power
- Mage → low HP, high Mage Power
- Rogue → medium HP, bonus starting gold

```
if hero_class == "Warrior":
    hp = 30
    gold = 10
elif hero_class == "Mage":
    hp = 18
    gold = 10
elif hero_class == "Rogue":
    hp = 22
    gold = 25
else:
    print("Unknown class - defaulting to Warrior")
    hp = 30
    gold = 10
```

HINT

Always include an else! What happens if someone types "warrior" in lowercase, or "Knight"? Handling unexpected input gracefully is a real skill — don't let your game crash because of a typo.

STEP 1.4 Print a character summary screen

After the class is chosen, print a clean summary showing the hero's name, class, HP, and gold, so the player can confirm their choices.

HINT

Use the empty `print("")` trick from Workshop 1 to add spacing and "-----" lines to make it look like a real game screen, like the Greeting Generator project.

Part 1 Checkpoint

You should be able to run the game, type a name and a class, and see a stat summary that's actually different depending on which class you picked.

Part 2 — The Village & First Choices

Now you'll build your first real "branching story" moment — where different choices lead to different `print()` outcomes, just like the personality quiz, but with consequences.

STEP 2.1 Print the village scene

Print a short scene description: the hero arrives at a village and hears two rumors. Then present 2–3 choices using a numbered menu (1, 2, 3).

```
print("You arrive at Maple Hollow village.")
print("An old woman tells you of a strange noise in the forest.")
print("")
print("1. Head into the forest immediately")
print("2. Rest at the inn first (+5 HP, costs 5 gold)")
print("3. Ask the villagers for more information")
choice = input("What do you do? (1/2/3): ")
```

STEP 2.2 Handle each choice with if / elif / else

Each option should actually do something: option 2 should change hp and gold variables, option 3 could just print extra lore, option 1 just moves the story forward.

HINT

This is the same skill as the grade calculator (mark >= 90, elif mark >= 75...) — except now your conditions check a menu number instead of a score.

STEP 2.3 Guard against bad gold math

If the player picks option 2 but doesn't have 5 gold, don't let gold go negative — print a message instead and don't apply the rest cost.

```
elif choice == "2":
    if gold >= 5:
        gold = gold - 5
        hp = hp + 5
        print("You rest at the inn. HP +5, Gold -5.")
    else:
        print("You don't have enough gold to rest!")
```

HINT

This is a nested if inside an elif — totally fine to do! It's just an if-statement living inside another condition's block.

Part 2 Checkpoint

Test all three choices separately (run the program 3 times). Confirm hp and gold update correctly and nothing goes negative.

Part 3 — The Forest Encounter (First Battle)

Time for your first real combat loop — this is where while loops shine, just like the Number Guessing Game and the password-checker loop.

STEP 3.1 Introduce the enemy

Print a short description of a forest enemy (e.g. a Goblin Scout) and give it its own stat, enemy_hp, e.g. 15.

STEP 3.2 Build a while loop that runs until someone reaches 0 HP

The loop should keep going while both `hp > 0` and `enemy_hp > 0` are true.

```
enemy_hp = 15

while hp > 0 and enemy_hp > 0:
    print("")
    print("Your HP:", hp, " | Goblin HP:", enemy_hp)
    action = input("1. Attack  2. Flee: ")

    if action == "1":
        enemy_hp = enemy_hp - 5
        hp = hp - 3
        print("You strike the goblin! It hits back.")
    elif action == "2":
        print("You flee back to the village.")
        break
    else:
        print("Invalid choice, you hesitate and take a hit!")
        hp = hp - 2
```

HINT

Two conditions joined with 'and' inside a while loop is new-ish, but it's just like the `>=` and `<=` comparisons from Workshop 2 — Python checks both sides and only continues if BOTH are true.

`break` is a new keyword here: it means 'immediately stop this loop.' It's perfect for a 'flee' option.

STEP 3.3 Print a win or lose message after the loop ends

After the while loop finishes, use `if/elif/else` to check who actually won (don't assume — the player could have fled, won, or lost).

```
if enemy_hp <= 0:
    print("You defeated the Goblin Scout!")
elif hp <= 0:
    print("You have been defeated... Game Over.")
else:
    print("You retreat, wounded but alive.")
```

HINT

This step is easy to forget! Without it, the game just silently stops with no explanation. Always tell the player what happened.

Day 1 Mini-Boss: The Goblin Scout

Play through the full game from the title screen to this fight at least 3 times: once where you win, once where you flee, and once where you let yourself lose. If all three feel right, Day 1 is done.

Day 2 — Inventory, the Gauntlet, and the Final Boss

DAY 2 GOAL: By the end of today, your game has a shop, a multi-round battle gauntlet, a real final boss fight, and more than one possible ending.

Part 4 — The Inventory & Shop

A simple shop screen is a great way to practice loops that repeat a menu until the player chooses to leave — like the To-Do List Manager, but for buying things.

STEP 4.1 Build a shop menu that repeats with a while loop

Print item options with prices (e.g. Healing Potion - 5 gold, Iron Sword - 15 gold) and let the player buy multiple times until they type "done".

```
shopping = True
potions = 0

while shopping == True:
    print("")
    print("Gold:", gold, " | Potions:", potions)
    print("1. Buy Potion (5 gold, +10 HP when used)")
    print("2. Leave shop")
    pick = input("Choose: ")

    if pick == "1":
        if gold >= 5:
            gold = gold - 5
            potions = potions + 1
            print("Bought a potion!")
        else:
            print("Not enough gold!")
    elif pick == "2":
        shopping = False
    else:
        print("Not a valid option.")
```

HINT

shopping = True / shopping = False as a loop condition is sometimes called a 'flag variable.' It's a clean way to control a loop without a number, similar to your password while loop from Workshop 2.

STEP 4.2 Let potions actually do something later

Make a note (a comment is fine!) that the potions count needs to be used during battles in Part 5 and 6 — e.g. an option to drink a potion to heal mid-fight.

 HINT

This is forward planning. You're not done with 'potions' yet — you're setting it up to pay off in your next battle. Real projects often have variables that get created in one section and used several sections later.

 Part 4 Checkpoint

Buy at least 2 potions, leave the shop, and print your gold/potions to confirm both updated correctly and the loop actually exits when you choose 'Leave shop.'

Part 5 — The Trial Gauntlet

This is a for loop project: the hero must survive a set number of waves before reaching the final boss, similar to the Quiz with Scoring System but turned into combat.

STEP 5.1 Use a for loop to run exactly 3 enemy waves

Use range() to repeat a wave 3 times, with each wave's enemy slightly stronger than the last.

```
for wave in range(1, 4):
    enemy_hp = 8 * wave    # gets harder each wave
    print("")
    print("--- Wave", wave, "---")
    print("A new enemy approaches! Enemy HP:", enemy_hp)
```

 HINT

range(1, 4) gives you 1, 2, 3 — just like the loop you used for grading in Workshop 2's range examples. wave will be 1, then 2, then 3, automatically.

STEP 5.2 Nest a while loop INSIDE the for loop for the actual fight

Inside each wave of the for loop, put a while loop just like Part 3's battle — but this time, also give the player the option to drink a potion to heal.

```
while hp > 0 and enemy_hp > 0:
    print("Your HP:", hp, " | Enemy HP:", enemy_hp, " | Potions:", potions)
    action = input("1. Attack  2. Drink Potion: ")

    if action == "1":
        enemy_hp = enemy_hp - 6
        hp = hp - (3 + wave)    # later waves hit harder
    elif action == "2":
        if potions > 0:
            hp = hp + 10
            potions = potions - 1
            print("You feel better!")
        else:
            print("No potions left!")
```

 HINT

A loop inside another loop is called a 'nested loop' — you've actually already done this pattern with if-inside-elif, it's the same idea.

Putting (3 + wave) instead of a flat number is a tiny touch that makes wave 3 genuinely scarier than wave 1 — small details like this make a game feel designed, not random.

STEP 5.3 Stop everything immediately if the hero dies mid-gauntlet

If hp <= 0 during any wave, you need to exit BOTH loops, not just the inner one. The cleanest way at your current level is a flag variable checked by the for loop too.

```
hero_alive = True

for wave in range(1, 4):
    if hero_alive == False:
        break # skip remaining waves entirely
    enemy_hp = 8 * wave
    while hp > 0 and enemy_hp > 0:
        ...
    if hp <= 0:
        hero_alive = False
```

 HINT

This 'flag variable' trick (hero_alive) is a real pattern professional developers use constantly to escape nested loops cleanly. You're not behind — this is genuinely a slightly advanced move, and you're handling it with tools you already know.

 Part 5 Checkpoint

Play through all 3 waves and confirm: dying on wave 2 correctly skips wave 3 (it should NOT keep going), and surviving all 3 waves moves you toward the boss.

Part 6 — The Final Boss

This is the moment everything pays off. The final boss fight should feel like the biggest, most dramatic version of everything you've already built.

STEP 6.1 Introduce the boss with a dramatic intro

Print a multi-line dramatic description of the final boss (e.g. The Shadow Wyrn) before the fight begins. Give it a noticeably bigger HP pool than anything before.

 HINT

This is just print() statements — but spend a little extra time on the writing here. The drama is free; it costs you nothing extra in code but makes the project feel finished.

STEP 6.2 Combine attack, potion, AND a special class move

Remember `hero_class` from Part 1? Use it now! Add a 3rd combat option that's different depending on whether the hero is a Warrior, Mage, or Rogue.

```
print("1. Attack  2. Drink Potion  3. Special Move")
action = input("Choose: ")

if action == "3":
    if hero_class == "Mage":
        boss_hp = boss_hp - 12
        print("You cast a fireball!")
    elif hero_class == "Warrior":
        boss_hp = boss_hp - 9
        hp = hp - 1    # warrior move costs a little HP
        print("You unleash a reckless power strike!")
    elif hero_class == "Rogue":
        if gold >= 10:
            gold = gold - 10
            boss_hp = boss_hp - 15
            print("You bribe a mercenary archer to help!")
        else:
            print("Not enough gold for your special move!")
```



HINT

This is the payoff for Part 1! All that work setting up `hero_class` wasn't wasted — this is exactly why developers plan their variables early.

Notice the Rogue's move is nested inside an if-check for gold, just like the inn-resting logic from Part 2. You're reusing a pattern you already know.

STEP 6.3 Build the full boss battle while loop

Put it all together: a while loop that runs while `hp > 0` and `boss_hp > 0`, offering Attack / Potion / Special Move each turn.



HINT

If you get stuck, go back and literally copy the structure from Part 5's wave-fight while loop. Re-using your own earlier code is not cheating — it's smart. Real developers copy their own patterns constantly.



Part 6 Checkpoint

Win the boss fight at least once and lose it at least once on purpose, to confirm both outcomes are handled and nothing crashes either way.

Part 7 — The Ending & Polish

A great project doesn't just stop — it ends on purpose, and it invites the player to try again.

STEP 7.1 Write at least 2 different ending messages

One ending for victory, one for defeat. Make the victory ending depend on `hero_class` for a personalized touch (e.g. "The bards will sing of the Mage who...").

STEP 7.2 Ask if the player wants to play again

At the very end, use `input()` to ask "Play again? (yes/no)" and use that to decide whether to print a goodbye message or restart.

HINT

A full restart-the-whole-game loop (wrapping everything in one big while loop) is possible but optional — for now it's completely fine to just print a friendly goodbye message and let the player re-run the program manually.

STEP 7.3 Final polish pass

Read through your whole file top to bottom and:

- Add comments above any section that isn't obvious at a glance
- Make sure spacing/`print("")` lines make the output easy to read
- Double check every `input()` that expects yes/no or a menu number has an else for unexpected typing

HINT

This polish pass is where good projects become great ones. It costs you 20 minutes and makes the difference between 'a script' and 'a project.'

Day 2 Final Checkpoint — YOU SHIPPED A GAME

Play your game start to finish, out loud if you can, like you're showing a friend. If it makes sense from title screen to ending without you needing to explain anything, you're done.

Bonus Quests (Optional Stretch Goals)

If you finish early or want an extra challenge, try any of these. They're optional — don't feel behind if you skip them.

Bonus 1 — Random enemy strength

Look up Python's random module (`import random`) and use `random.randint()` to make enemy HP slightly different each playthrough instead of a fixed number.

HINT

This is the only tool in this whole project NOT from your workshops — a perfect, low-risk first step into exploring documentation on your own.

Bonus 2 — A second shop visit before the boss

Let the player return to the shop after the gauntlet (Part 5) but before the boss (Part 6), so good gold management actually matters.

Bonus 3 — A title screen menu

Before character creation, add a menu: "1. Start New Game 2. How to Play 3. Quit" — and write a short How to Play screen explaining the combat options.

Bonus 4 — Track and display total turns taken

Add a turn counter variable that goes up by 1 every time the player picks a combat action, and print the total at the end ("You triumphed in 14 turns!").

Appendix — Your Core Variable Tracker

Keep this table updated as you build. If you ever feel lost in your own code, this table is your map back.

Variable	Type	Where it's used
hero_name	string	Set in Part 1.1, used in print messages throughout
hero_class	string	Set in Part 1.2, checked in Part 1.3 and Part 6.2
hp	integer	Set in Part 1.3, changed in Parts 2, 3, 5, 6
gold	integer	Set in Part 1.3, changed in Parts 2, 4, 6
potions	integer	Set in Part 4.1, used in Part 5.2
enemy_hp / boss_hp	integer	Reset at the start of each fight (Parts 3, 5, 6)
hero_alive	boolean	Flag variable introduced in Part 5.3

You built a multi-system game using nothing but `print()`, variables, `input()`, `if/elif/else`, and `for/while` loops. That's the whole craft — everything else is just more of this, at a bigger scale. Go show it off. 🏆